

Unit 2 : Advanced Java Concept

2.2 Multithreading in Java :

Introduction

- **Multithreading** is a programming technique in Java that allows a program to execute **multiple threads concurrently** within a single process.
- A **thread** is the smallest unit of execution within a process.
- Java provides built-in support for multithreading through the **java.lang package**, making it easier to build efficient and responsive applications.

➤ **Real-Life Examples**

- Web servers handling multiple client requests
- Banking systems processing transactions simultaneously

➤ **Why Multithreading?**

Benefits:

1. Improves CPU utilization
2. Faster execution
3. Better responsiveness
4. Efficient resource sharing
5. Parallel execution
6. Improves performance of multi-core systems

➤ Ways to Create Threads in Java:

Java supports multithreading using:

1. **Thread class** (java.lang.Thread)
2. **Runnable interface** (java.lang.Runnable)

1. Thread Class

- The **Thread class** in Java is used to create and manage threads.
- It is part of the **java.lang package** and provides constructors and methods to perform multithreading.
- A thread represents a separate path of execution within a program.
- **Definition**

“**Thread class** allows a program to run multiple threads concurrently by creating a new thread of execution.”

- **Package**

java.lang.Thread

➤ Syntax to Create Thread Using Thread Class

Steps:

1. Extend Thread class
2. Override run() method
3. Create object
4. Call start()

➤ Example 1:

```
class mythread extends Thread
{

    public void run()
    {
```

```

        System.out.println("Thread running");
    }

    public static void main(String[] args)
    {
        mythread t = new mythread();
        t.start();
    }
}

```

Output:

Thread running

➤ Explanation:

How It Works:

1. start() creates a new thread
2. JVM calls run() method
3. Code inside run executes concurrently

➤ Important Methods of Thread Class

1. start()

- Starts a new thread and calls run() internally.
- t.start();

2. run()

- Contains the code executed by thread.

```

public void run()
{
    // task
}

```

3. sleep()

- Pauses thread execution for specified time.
- Thread.sleep(1000);
- □ Time in milliseconds

4. join()

- Makes current thread wait until another thread finishes.
- t.join();

5. yield()

- Pauses current thread to allow others to execute.
- Thread.yield();

6. getName() / setName()

- Gets or sets thread name.
- t.setName("Worker");
System.out.println(t.getName());

7. getPriority() / setPriority()

- Gets or sets thread priority.
- t.setPriority(10);

8. isAlive()

- Checks whether thread is running.
- t.isAlive();

9. currentThread()

- Returns currently executing thread.
- Thread.currentThread();

2. Implementing Runnable Interface:

➤ What is Runnable?

- Runnable is an **interface** in Java used to create a thread.
- It has only **one method** → **run()**.
- We write the task inside run().

➤ Why use Runnable?

- Allows a class to extend another class
- Better for sharing task between threads
- Recommended way to create thread

➤ Steps to Implement Runnable

1. Create a class
2. Use implements Runnable
3. Override run() method
4. Create object of class
5. Pass object to Thread
6. Call start()

➤ Example 2:

```
class MyRunnable implements Runnable
{

    public void run()
    {
        System.out.println("Thread is running");
    }

    public static void main(String[] args)
```

```
{  
  
    MyRunnable obj = new MyRunnable();  
    Thread t = new Thread(obj);  
    t.start();  
}  
}
```

Output:

Thread is running

➤ Explanation:

How It Works

Runnable = task
Thread = worker

Thread takes task and executes it.

Flow

Create class → implement Runnable → write run()
→ create object → pass to Thread → start()

Real Life Example

Runnable = Work instruction
Thread = Worker

Worker performs task written in instruction.

2.2.1 Thread Life Cycle:

- In Java, a thread goes through different stages during its execution.
- These stages are defined in the Java Programming Language under the Thread class.
- A thread life cycle describes the various states a thread enters from its creation to termination.

States of Thread Life Cycle

1. New State

- A thread is in the **New** state when it is created but not yet started.
- It is created using the Thread class or by implementing the Runnable interface.
- The thread will not execute until the start() method is called.\
- Example:

```
Thread t = new Thread();
```

Here, the thread is in the **New** state.

2. Runnable State:

- When start() method is called, the thread enters the **Runnable** state.
- The thread is ready to run and waiting for CPU time.
- It depends on the thread scheduler to move to Running state.
- t.start();

3. Running State

- The thread is in **Running** state when the CPU starts executing the run() method.
- Only one thread runs at a time.
- After execution, it may move to:Blocked, Waiting, Terminated state.

4. Blocked State:

- The thread will be in blocked state **when it is trying to acquire a lock** but currently the lock is acquired by the other thread.
- The thread will move from the blocked state to runnable state when it acquires the lock.

5. Waiting State

- The thread will be in waiting state **when it calls wait() method or join() method.**
- It will move to the runnable state when other thread will notify or that thread will be terminated.

6. Timed Waiting State

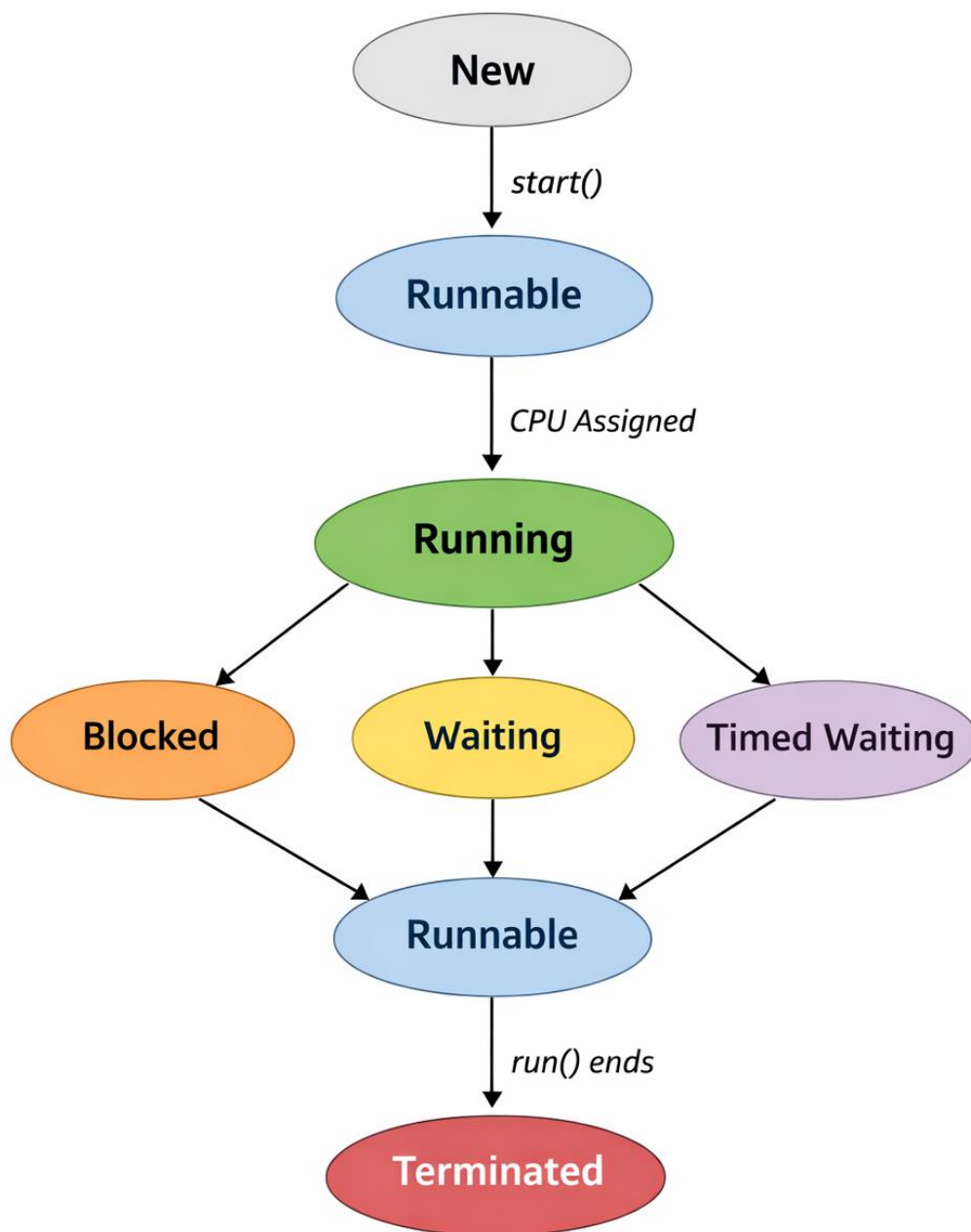
- A thread waits for a specified amount of time.
- Methods: 1. sleep(time), 2. wait(time), 3. join(time)
- Example:

Thread.sleep(2000);

Thread sleeps for 2 seconds.

7. Terminated (Dead) State

- A thread enters **Terminated** state when:
- run() method completes
- Thread finishes execution or stops.
- An exception occurs: A terminated thread cannot be restarted.



2.2.2 Thread Priorities

1. Introduction to Thread Priority

- In Java, every thread has a **priority** that helps the **thread scheduler** decide which thread should run first.
- When multiple threads are ready to run, the **thread with higher priority gets more chance to execute.**
- **Example:**
If two threads are running:

Thread A → Priority 8

Thread B → Priority 4

Then **Thread A will usually run before Thread B.**

2. Priority Range in Java

Thread priority values range from **1 to 10**.

Priority Level	Constant Name	Value
Minimum Priority	MIN_PRIORITY	1
Normal Priority	NORM_PRIORITY	5
Maximum Priority	MAX_PRIORITY	10

Java provides these constants in the **Thread class**.

Example:

Thread.MIN_PRIORITY
Thread.NORM_PRIORITY
Thread.MAX_PRIORITY

3. Default Priority

- When a thread is created in Java:
- It gets the **same priority as the thread that created it.**
- The **main thread priority is 5 (NORM_PRIORITY).**
- Example:

```
Thread t = new Thread();  
System.out.println(t.getPriority());
```

Output:

5

4. Setting Thread Priority

- Thread priority can be set using:

```
setPriority(int priority)
```

Example Program

```
class Test extends Thread
```

```
{  
    public void run()  
    {
```

```
        System.out.println(Thread.currentThread().getName());  
    }  
}
```

```
public class Demo
```

```
{  
    public static void main(String args[])  
    {
```

```
Test t1 = new Test();
Test t2 = new Test();

t1.setPriority(10);
t2.setPriority(3);

t1.start();
t2.start();
}
}
```

Output:

Thread-0

Thread-1

Explanation:

t1 priority = 10 (High)

t2 priority = 3 (Low)

Thread scheduler gives **more chance to t1**.

5. Getting Thread Priority

- To check priority of a thread:

getPriority()

- Example:

```
System.out.println(Thread.currentThread().getPriority());
```

- This prints the **current thread priority**.

Advantages of Thread Priority

1. Helps manage **important tasks first**.
2. Improves **performance of multitasking programs**.
3. Useful in **real-time applications**.

Disadvantages

1. Execution order is **not guaranteed**.
2. Depends on **operating system scheduler**.

Short Definition:

Thread priority in Java is a value between **1 and 10** that helps the **thread scheduler decide which thread should run first**.

2.2.3 Synchronizing Threads

1. Meaning

- **Synchronization** means **controlling multiple threads so that they do not access shared resources at the same time.**
- When many threads work on the **same data**, problems like **data inconsistency** may occur.
- Synchronization ensures: Only **one thread accesses the resource at a time.**

2. Why Synchronization is Needed

- Example problem: Two threads updating the **same bank account balance.**

Without synchronization:

Thread 1 → withdraw ₹500

Thread 2 → withdraw ₹500

Balance may become **incorrect.**

Synchronization prevents this issue.

3. Using synchronized Keyword

Java provides the synchronized **keyword** to achieve synchronization.

Example:

```
class Test
```

```
{  
    synchronized void show()  
    {  
        System.out.println("Thread is running");  
    }  
}
```

```
class MyThread extends Thread
```

```
{  
    Test t;  
  
    MyThread(Test t)  
    {  
        this.t = t;  
    }  
  
    public void run()  
    {  
        t.show();  
    }  
}
```

```
public class DemoS
```

```
{  
    public static void main(String args[])  
    {
```

```
Test obj = new Test();

MyThread t1 = new MyThread(obj);
MyThread t2 = new MyThread(obj);

t1.start();
t2.start();
}
}
```

Output:
Thread is running
Thread is running

Explanation

show() method is **synchronized**.

Only **one thread can execute the method at a time**.

The second thread waits until the first thread finishes.

2.2.4 Inter-Communication of Threads

Meaning:

- **Inter-Thread Communication** is a mechanism in Java where **threads communicate with each other to coordinate their work.**
- It is used when:

One thread must **wait for another thread to complete some task.**

- **Example:**
One thread **produces data** and another thread **uses that data.**

Methods Used in Inter-Thread Communication

Java provides three important methods:

Method	Purpose
wait()	Makes the thread wait
notify()	Wakes up one waiting thread
notifyAll()	Wakes up all waiting threads

These methods are defined in the **Object class.**

Example:

```
class A extends Thread
{
    public void run()
    {
        synchronized(this)
        {
            System.out.println("Child thread");
            notify();
        }
    }
}

class Test {
    public static void main(String args[]) throws Exception
    {
        A t = new A();

        synchronized(t)
        {
            t.start();
            t.wait();
        }

        System.out.println("Main thread");
    }
}
```

Output

Child thread
Main thread

Explanation

1. Create Thread Class

class A extends Thread :

A new class **A** is created.

It **extends Thread**, so it becomes a **thread class**.

2. Run Method

public void run() :

run() method contains the **code that the thread will execute**.

3. Synchronization

synchronized(this):

synchronized means **only one thread can access this block at a time**.

this : means the **current object**.

4. Notify Method

notify();

notify() **wakes up another thread** that is waiting.

5. Main Method

public static void main(String args[]) throws Exception :

- throws Exception is written because the program uses the wait() **method**, and wait() can cause a **checked exception** called **InterruptedException**.
- So Java requires that we **handle or declare the exception**.
- wait() may throw **InterruptedException**.
- Therefore we must either:
 1. Use **try-catch**, or
 2. Write **throws Exception** in the method.

6. Synchronize Object

synchronized(t)

Reason: wait() must be called inside synchronized block

In Java, the wait() **method can only be used when the thread owns the object's lock.**

7. Start Thread

t.start();

Starts the **child thread**.

It automatically calls the **run() method**.

8. Wait Method

t.wait(); :The **main thread waits** until another thread calls notify().

Important Points

1. wait(), notify(), notifyAll() must be used inside **synchronized block or method**.
2. They help threads **coordinate execution**.

2.3 Exception Handling

1. Introduction

- **Exception Handling** is a mechanism in Java used to **handle runtime errors** so that the **normal flow of the program is maintained**.
- An **exception** is an **unexpected event that occurs during program execution and disrupts the normal flow of instructions**.
- Example situations where exceptions occur:

Dividing a number by zero

Accessing an invalid array index

Opening a file that does not exist

- Without exception handling, the program **terminates abnormally**.

Example Without Exception Handling:

```
class Test
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 0;
        int c = a/b;

        System.out.println(c);
    }
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero

The program **stops immediately** because the exception is not handled.

Example With Exception Handling:

```
class Test1
{
    public static void main(String args[])
    {
        try
        {
            int a = 10;
            int b = 0;
            int c = a/b;
        }
        catch(ArithmeticException e)
        {
            System.out.println("Cannot divide by zero");
        }

        System.out.println("Program continues...");
    }
}
```

Output

Cannot divide by zero

Program continues...

Here the exception is **handled using try-catch**, so the program continues.

2.3.1 Types of Exceptions

In Java, exceptions are mainly divided into **two types**.

Type	Description
Checked Exception	Checked at compile time
Unchecked Exception	Occurs at runtime

1. Checked Exceptions:

- Checked exceptions are **checked by the compiler at compile time**.
- The programmer **must handle them** using try-catch or throws.
- **Examples:**

IOException

FileNotFoundException

ClassNotFoundException

Example

```
import java.io.*;

class Test3
{
    public static void main(String args[])
    {
        try
        {
            FileReader f = new FileReader("abc.txt");
        }
    }
}
```

```
    }  
    catch(Exception e)  
    {  
        System.out.println("File not found");  
    }  
}  
}
```

Output:

File not found

Explanation:

The program tries to open **abc.txt**.

If the file does not exist, **FileNotFoundException** occurs.

This is a **Checked Exception** because the **compiler checks it**.

2. Unchecked Exceptions:

- Unchecked exceptions occur **during program execution (runtime)**.
- They are **not checked by the compiler**.
- **Examples:**

ArithmeticException

ArrayIndexOutOfBoundsException

Example

```
class Test4
{
    public static void main(String args[])
    {
        int a = 10;
        int b = 0;

        int c = a / b;

        System.out.println(c);
    }
}
```

Output:

Exception in thread "main" java.lang.ArithmeticException: / by zero

2.3.2 Keywords Used in Exception Handling

Java provides **five important keywords** for handling exceptions.

Keyword	Description
try	Contains code that may cause an exception
catch	Handles the exception
finally	Executes whether exception occurs or not
throw	Used to throw an exception manually

1. try:

- The **try block** contains code that may generate an exception.

Example

```
try
{
    int a = 10/0;
}
```

2. catch:

- The **catch block** handles the exception that occurs in the try block.

Example

```
try
{
    int a = 10/0;
}
catch(ArithmeticException e)
{
    System.out.println("Error occurred");
}
```

3. finally:

- The **finally block always executes** whether an exception occurs or not.
- It is used for **cleanup operations** like closing files.

Example

```
try
{
    int a = 10/0;
}
catch(Exception e)
{
    System.out.println("Exception handled");
}
finally
{
    System.out.println("Finally block executed");
}
```

Output

Exception handled
Finally block executed

4. throw:

- The **throw keyword is used to explicitly throw an exception.**

Example

```
class Test
{
    public static void main(String args[])
    {
        throw new ArithmeticException("Manual Exception");
    }
}
```

Output

ArithmeticException: Manual Exception

Example on try, catch, throw, finally:

```
class Test5
{
    public static void main(String args[])
    {
        try
        {
            int a = 10;
            int b = 0;

            if(b == 0)
            {
                throw new ArithmeticException("Cannot
divide by zero");
            }

            int c = a/b;
            System.out.println(c);
        }
        catch(ArithmeticException e)
        {
            System.out.println("Exception caught: " +
e.getMessage());
        }
        finally
        {
            System.out.println("Finally block always
executes");
        }
    }
}
```

```
    }  
  }  
}
```

Output:

Exception caught: Cannot divide by zero
Finally block always executes

Explanation :

1. try block:

Contains code that may cause an error.

2. throw

Manually throws an exception when `b == 0`.

3. catch block

Handles the exception and prints the message.

4. finally block

Always executes whether exception occurs or not.

2.3.3 User Defined Exception:

a) Write a java program to demonstrate how to create user-defined exception.
[Previous year Question Paper: 5 Marks]

- Java allows programmers to **create their own custom exceptions**.
- These are called **User Defined Exceptions**.
- A user-defined exception is created by **extending the Exception class**.

Steps to Create User Defined Exception

1. Create a class that **extends Exception**.
2. Create a **constructor** to display the error message.
3. Use the **throw keyword** to throw the exception.
4. Handle it using **try-catch block**.

Syntax

```
class MyException extends Exception
{
    MyException(String message)
    {
        super(message);
    }
}
```

super(message) is used to pass the message to the Exception class.

Example

```
class MyException extends Exception
{
    MyException(String msg)
    {
        super(msg);
    }
}
```

```

class Test6
{
    public static void main(String args[])
    {
        try
        {
            throw new MyException("This is custom
exception");
        }
        catch(MyException e)
        {
            System.out.println(e.getMessage());
        }
    }
}

```

Output

This is custom exception

Explanation:

1. MyException is a **user defined exception class** that extends the Exception class.
2. The constructor passes the **error message** using super(msg).
3. In main(), the program **throws the custom exception** using throw.
4. The catch block **handles the exception** and prints the message using e.getMessage().

➤ **Short Definitions for Exam**

Exception:

An exception is an **event that occurs during program execution and disrupts the normal flow of instructions.**

Exception Handling:

Exception handling is a mechanism in Java used to **handle runtime errors and maintain normal program execution.**

User Defined Exception:

A user-defined exception is a **custom exception created by the programmer by extending the Exception class.**

2.1 Introduction to Collection Framework:

- The **Collection Framework** in Java is a set of **classes and interfaces** used to **store, manage, and manipulate groups of objects** dynamically.
- Before the collection framework, Java used **arrays**, but arrays have a **fixed size**.
- Collections provide **dynamic size**, better performance, and many useful methods for handling data.

Definition

The **Java Collection Framework (JCF)** is a unified architecture that provides **interfaces, classes, and algorithms** to work with a group of objects.

Advantages

1. **Dynamic size** – Collections can grow or shrink automatically.
2. **Reusable data structures** – Ready-made classes like lists, sets, and maps.
3. **Easy data manipulation** – Methods for searching, sorting, inserting, and deleting.
4. **Improves programming efficiency.**

➤ Main Interfaces in Collection Framework

1. **List** – Stores ordered elements and allows duplicates.
Example: ArrayList, LinkedList

2. **Set** – Stores unique elements (no duplicates).
Example: HashSet, TreeSet

3. **Queue** – Follows FIFO (First In First Out).
Example: PriorityQueue, LinkedList

4. **Map** – Stores data in **key–value pairs**.
Example: HashMap, TreeMap

2.1.1 ArrayList, Vector, Set, Map, Hashing (Java Collections)

1. ArrayList

- **ArrayList** is a class in Java that is part of the **Collection Framework**.
- It is used to store **dynamic arrays** that can grow or shrink automatically.

Features:

1. Size is **dynamic** (not fixed like arrays).
2. Allows **duplicate elements**.
3. Maintains **insertion order**.
4. Elements can be accessed using **index**.
5. Not synchronized (faster but not thread-safe).

Syntax:

```
import java.util.ArrayList;
```

```
ArrayList<String> list = new ArrayList<>();
```

Where:

1. import java.util.ArrayList;

This line allows the program to **use the ArrayList class**.

2. ArrayList<String> list = new ArrayList<>();

Create an ArrayList named list that stores String elements.

Breakdown:

ArrayList → Class used to create a dynamic list.

<String> → Specifies the type of elements stored in the list (only strings).

list → Variable name (reference to the ArrayList object).

new → Keyword used to create an object.

ArrayList<>() → Constructor that creates the ArrayList.

Important Methods:

Method	Description
add()	Adds element
get()	Retrieves element
remove()	Removes element
size()	Returns number of elements

Example 1: ArrayList

```
import java.util.ArrayList;

class Test
{
    public static void main(String args[])
    {
        ArrayList<String> list = new ArrayList<>();

        list.add("Apple");
        list.add("Mango");
        list.add("Banana");

        System.out.println(list);
    }
}
```

Output:

[Apple, Mango, Banana]

Example 2: Arraylist

b) Write a Java program to implement arraylist with function to add,remove & sort member.

[5]

[Previous year question Paper]

```
import java.util.ArrayList;
import java.util.Collections;

class ArrayListDemo
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();

        // Add elements
        list.add("Sahil");
        list.add("Rahul");
        list.add("Amit");
        list.add("Neha");

        System.out.println("ArrayList after adding
elements:");
        System.out.println(list);

        // Remove element
        list.remove("Rahul");

        System.out.println("ArrayList after removing
element:");
```

```
System.out.println(list);

// Sort elements
Collections.sort(list);

System.out.println("ArrayList after sorting:");
System.out.println(list);
}
}
```

Output:

ArrayList after adding elements:
[Sahil, Rahul, Amit, Neha]

ArrayList after removing element:
[Sahil, Amit, Neha]

ArrayList after sorting:
[Amit, Neha, Sahil]

Explanation:

1. `ArrayList<String> list = new ArrayList<>();` → Creates a dynamic list of strings.
2. `list.add()` → Adds elements to the list.
3. `list.remove()` → Removes an element from the list.
4. `Collections.sort()` → Sorts the elements in ascending order.

The program shows how to **add, remove, and sort elements in an ArrayList**.

2. Vector

- **Vector** is similar to ArrayList but it is **synchronized**.
- This means it is **thread-safe** and can be used in **multi-threaded environments**.

Features:

1. Dynamic array.
2. Allows **duplicate elements**.
3. Maintains **insertion order**.
4. **Synchronized** (slower than ArrayList).

Syntax

```
Vector<Integer> v = new Vector<>();
```

Example

```
import java.util.*;
class Demo
{
    public static void main(String args[])
    {
        Vector<Integer> v = new Vector<>();

        v.add(10);
        v.add(20);
        v.add(30);

        System.out.println(v);
    }
}
```

Output:

```
[10, 20, 30]
```

3. Set

- **Set** is an interface in Java Collection Framework that stores **unique elements**.

Features

1. **No duplicate elements allowed**
2. Does not maintain insertion order (in most implementations)
3. Used when **unique data** is required

Types of Set

HashSet

LinkedHashSet

TreeSet

Example:

```
import java.util.*;

class DemoSet
{
    public static void main(String args[]) {
        Set<String> set = new HashSet<>();

        set.add("Java");
        set.add("Python");
        set.add("Java"); // duplicate

        System.out.println(set);
    }
}
```

Output:

[Java, Python]

Explanation:

```
Set<String> set = new HashSet<>();
```

Set – An interface in Java collection framework.

<String> – Generic type, meaning the set will store **String values only**.

set – Variable name.

new HashSet<>() – Creates an object of **HashSet class**.

HashSet – A collection that **does not allow duplicate elements**.

Duplicate values are **automatically removed**.

4. Map

- **Map** is used to store data in **key–value pairs**.
- Example:

RollNo → Name

101 → Sahil

102 → Rahul

Features:

1. Stores **key and value pairs**
2. **Keys must be unique**
3. Values can be duplicate
4. Does not extend Collection interface

Common Map Classes:

1. HashMap
2. LinkedHashMap
3. TreeMap

Syntax:

```
Map<Integer, String> map = new HashMap<>();
```

Where :

Map → An interface from java.util used to store **key–value pairs**.

<Integer, String> → Generic types where **Integer is the key** and **String is the value**.

map → Variable (reference) name.

new HashMap<>() → Creates an object of the **HashMap class**, which implements the Map interface.

Example:

```
import java.util.*;

class DemoMap
{
    public static void main(String args[]) {
        Map<Integer,String> map = new HashMap<>();

        map.put(1, "Sahil");
        map.put(2, "Rahul");

        System.out.println(map);
    }
}
```

Output:

{1=Sahil, 2=Rahul}

Explanation :

- This Java program demonstrates the use of **Map and HashMap**.
- The Map<Integer, String> stores **key–value pairs** where **Integer is the key** and **String is the value**.
- A HashMap object is created and two entries (1, "Sahil") and (2, "Rahul") are added using the put() method.
- Finally, System.out.println() prints all the key–value pairs stored in the map.

5. Hashing

- **Hashing** is a technique used to **convert data into a unique hash value (hash code)** for faster searching.
- Hashing is mainly used in: **HashMap, HashSet, Hashtable**

How Hashing Works:

1. A **hash function** converts a key into a **hash code**.
2. The hash code determines the **storage location**.
3. This allows **very fast data retrieval**.

Example:

```
HashMap<Integer,String> map = new HashMap<>();  
map.put(101,"Sahil");  
map.put(102,"Rahul");
```

Here,

101 and 102 are **keys**

Hash function converts them into **memory index**

Summary Table

Collection	Allows Duplicate	Maintains Order	Key-Value
ArrayList	Yes	Yes	No
Vector	Yes	Yes	No
Set	No	No	No
Map	Value yes / Key no	Depends	Yes

[Previous year Question Paper May 2025]

Q.2 a) Create a thread to display prime numbers between 1 to 500 each number will display after 3 seconds.

[5]

Program Code:

```
class MyThread extends Thread
{
    public void run()
    {
        for(int i=2;i<=500;i++)
        {
            int flag=0;

            for(int j=2;j<i;j++)
            {
                if(i%j==0)
                {
                    flag=1;
                    break;
                }
            }

            if(flag==0)
            {
                System.out.println(i);

                try{
                    Thread.sleep(3000); // 3 seconds delay
                }
            }
        }
    }
}
```

```

        }
        catch(Exception e){}
    }
}
}

class DemoPrime
{
    public static void main(String args[])
    {
        MyThread t = new MyThread();
        t.start();
    }
}

```

Output:

2
 3
 5
 7
 11
 13
 17
 19
 23
 29
 ...

Explanation:

- This program creates a **thread by extending the Thread class**.
- The run() method checks numbers from **2 to 500** to find **prime numbers**.
- If a number is prime, it is printed using System.out.println().
- After printing each prime number, the thread pauses for **3 seconds using Thread.sleep(3000)**.
- The thread starts execution using the start() **method** in the main() function.

B) Write a java program to demonstrate how to create user-defined exception. [Previous year Question Paper: 5 Marks]

- Java allows programmers to **create their own custom exceptions**.
- These are called **User Defined Exceptions**.
- A user-defined exception is created by **extending the Exception class**.

Steps to Create User Defined Exception

5. Create a class that **extends Exception**.
6. Create a **constructor** to display the error message.
7. Use the **throw keyword** to throw the exception.
8. Handle it using **try-catch block**.

Syntax

```
class MyException extends Exception
{
    MyException(String message)
    {
        super(message);
    }
}
```

super(message) is used to pass the message to the Exception class.

Example

```
class MyException extends Exception
{
```

```

    MyException(String msg)
    {
        super(msg);
    }
}

class Test6
{
    public static void main(String args[])
    {
        try
        {
            throw new MyException("This is custom
exception");
        }
        catch(MyException e)
        {
            System.out.println(e.getMessage());
        }
    }
}

```

Output

This is custom exception

Explanation:

5. MyException is a **user defined exception class** that extends the Exception class.
6. The constructor passes the **error message** using super(msg).

7. In main(), the program **throws the custom exception** using throw.
8. The catch block **handles the exception** and prints the message using e.getMessage().

c) Write a Java program to implement arraylist with function to add,remove & sort member.

[5]

[Previous year question Paper]

```
import java.util.ArrayList;
import java.util.Collections;

class ArrayListDemo
{
    public static void main(String[] args)
    {
        ArrayList<String> list = new ArrayList<>();

        // Add elements
        list.add("Sahil");
        list.add("Rahul");
        list.add("Amit");
        list.add("Neha");

        System.out.println("ArrayList after adding
elements:");
        System.out.println(list);

        // Remove element
        list.remove("Rahul");
```



```
        System.out.println("ArrayList after removing  
element:");  
        System.out.println(list);  
  
        // Sort elements  
        Collections.sort(list);  
  
        System.out.println("ArrayList after sorting:");  
        System.out.println(list);  
    }  
}
```

Output:

ArrayList after adding elements:
[Sahil, Rahul, Amit, Neha]

ArrayList after removing element:
[Sahil, Amit, Neha]

ArrayList after sorting:
[Amit, Neha, Sahil]

Explanation:

5. `ArrayList<String> list = new ArrayList<>();` → Creates a dynamic list of strings.
6. `list.add()` → Adds elements to the list.
7. `list.remove()` → Removes an element from the list.
8. `Collections.sort()` → Sorts the elements in ascending order.

The program shows how to **add, remove, and sort elements in an ArrayList**.

Q.2 Differentiate between checked and unchecked exception.

d) [Previous Year Question Paper- 5 marks]

No.	Checked Exception	Unchecked Exception
1	Checked exceptions are checked by the compiler at compile time .	Unchecked exceptions are checked at runtime .
2	Checked exceptions must be handled using try-catch or throws.	Unchecked exceptions do not require mandatory handling .
3	Checked exceptions usually occur due to external problems like file or database issues.	Unchecked exceptions usually occur due to programming mistakes .
4	If a checked exception is not handled, the program will not compile .	If an unchecked exception is not handled, the program compiles but fails at runtime .
5	Checked exceptions are subclasses of Exception (except runtime exceptions).	Unchecked exceptions are subclasses of RuntimeException.
6	Checked exceptions help in improving program reliability because they must be handled.	Unchecked exceptions usually indicate logical errors in the program .
7	Checked exceptions commonly occur in input-output operations .	Unchecked exceptions occur during incorrect calculations or null references .
8	Example: IOException occurs during file handling.	Example: ArithmeticException occurs when dividing by zero.
9	Another example of checked exception is SQLException.	Another example of unchecked exception is NullPointerException.
10	These exceptions are checked by the compiler before program execution .	These exceptions are identified only when the program runs .